TripSync

Wspólny wyjazd. Jeden plan.

Dokumentacja projektu Android

Wykonawcy: Brajan Sieroń, Damian Panońko, Patryk Dawczak.

Wersja aplikacji: 1.2.1 (kod 4) | **Backend:** API 1.2.0 | **Data:** 06.11.2026.

Wydanie do oddania uwzględnia poprawkę komunikatów połączenia z API (commit 1b4c1a9).

1. Cel i zakres aplikacji

TripSync wspiera organizację wyjazdów niewielkich grup znajomych. Łączy wspólny harmonogram, zadania i pakowanie, propozycje z głosowaniem oraz koszty i rozliczenia. Celem jest zebranie informacji o podróży w jednym miejscu i umożliwienie pracy także podczas utraty połączenia.

Zakres obejmuje konta i role, zaproszenia, CRUD wyjazdów i ich elementów, budżety, kategorie wydatków, zapis spłat, historię zmian, mapę miejsc, wyszukiwanie miejscowości i prognozę na 7 dni. Zmiany zapisują się lokalnie i synchronizują z serwerem; konflikty można rozstrzygnąć w aplikacji.

Gość korzysta z demo lub zakłada konto. Uczestnik współtworzy dane wyjazdu. Organizator zarządza dodatkowo wyjazdem, zaproszeniami, uczestnikami i budżetem. Każdy wyjazd ma jedną walutę (PLN, EUR lub USD). Spłaty są deklaracjami użytkowników. Kafelki mapy i nowa prognoza wymagają internetu.

Materiały i kod źródłowy

Repozytorium Git: github.com/Yamoteusz/TripSync

Aplikacja w paczce: TripSync.apk, Android 8.0 lub nowszy, podpis debug do prezentacji. Ten dokument zawiera wymagane opisy, diagramy, wyniki testów i zrzuty ekranu.

Spis treści

1. Cel i zakres · 2. Wymagania funkcjonalne · 3. Wymagania niefunkcjonalne · 4. Architektura · 5. Przypadki użycia · 6. Bazy danych · 7. Technologie · 8. Instalacja i uruchomienie · 9. Testy · 10. Zrzuty ekranu.

2. Wymagania funkcjonalne

CRUD oznacza dodawanie, odczytywanie, edycję i usuwanie danych. Poniższe kryteria określają oczekiwane zachowanie aplikacji.

ID	Wymaganie	Kryterium odbioru
F01	Rejestracja, logowanie, wylogowanie	Walidacja, sesja lokalna i unieważnienie tokenu przy wylogowaniu online
F02	CRUD wyjazdów	Nazwa, cel, daty, waluta i opis; edycja/usunięcie przez organizatora
F03	Uczestnicy i zaproszenia	Kod 12 znaków lub link; rotacja kodu i kontrola dostępu
F04	CRUD harmonogramu	Data w terminie wyjazdu, godzina, miejsce, kolejność i wykonanie
F05	Propozycje i głosy	CRUD propozycji, jeden własny głos na propozycję, cofnięcie i ponowne oddanie
F06	Propozycja do planu	Jedna wersjonowana operacja zmienia pomysł w punkt, usuwa głosy i nie tworzy duplikatów
F07	Zadania i pakowanie	CRUD, kategoria, osoba, wykonanie i opcjonalne przypomnienie
F08	Wydatki	CRUD, płatnik, data, uczestnicy, podział równy lub kwoty własne
F09	Rozliczenia	Dokładność groszy, suma sald zero i propozycje przelewów
F10	Offline i trwałość	Lokalny zapis bez sieci, ponowny odczyt po uruchomieniu
F11	Synchronizacja	Kolejka, ponowienia bez duplikatów, wersje oraz wybór wersji przy konflikcie
F12	Powiadomienia lokalne	Przypomnienie punktu/zadania, anulowanie po wykonaniu/usunięciu
F13	Stan i ustawienia	Zachowanie formularza po odtworzeniu aktywności, motyw i adres API w DataStore
F14	Obsługa błędów	Walidacja, komunikaty sieci/serwera i stan odrzuconych operacji
F15	Demo i udostępnianie	Edytowalny przykład offline oraz udostępnienie tekstowego planu
F16	Budżet	CRUD limitu i kategorii, limit kategorii 0 i potwierdzanie usunięcia przez organizatora
F17	Kategorie i lista kosztów	Sześć kategorii, zestawienie, filtr, wyszukiwanie i sortowanie
F18	Spłaty	CRUD własnych spłat; salda uwzględniają spłatę częściową i nadpłatę
F19	Historia zmian	Autorytatywne wpisy serwera z autorem i czasem, odczyt bez edycji klienta
F20	Miejsca	CRUD współrzędnych, mapa wspólna i otwarcie w zewnętrznych mapach
F21	Wyszukiwanie	Geokodowanie miejscowości i zapis wyniku jako miejsce
F22	Prognoza	7 dni, temperatury, opis oraz szansa opadów
F23	Cache pogody	Ostatni wynik i daty dostępne po błędzie sieci
F24	Daty i godziny	Kalendarz i zegar 24-godzinny; w bazie ISO, prezentacja po polsku
F25	Ochrona formularza	Potwierdzenie odrzucenia zmian w edytorze głównym, budżecie, spłacie i miejscu
F26	Widoki wyjazdów i planu	Wyszukiwanie, aktywne/zakończone wyjazdy, plan według dni

3. Wymagania niefunkcjonalne

ID	Wymaganie	Realizacja i granice
N01	Material 3 i responsywność	Compose, motywy, przewijanie, maksymalna szerokość i kompaktowe pickery; szeroka weryfikacja tabletów pozostaje do wykonania
N02	Architektura i modularność	MVVM, foldery ui/domain/data i odrębny backend; jedna aplikacja Gradle, nie pełna Clean Architecture
N03	Asynchroniczność	Coroutines, Flow, suspend Room/Retrofit i I/O poza głównym wątkiem
N04	Zmiana konfiguracji	ViewModel i rememberSaveable; test odtworzenia aktywności, bez pełnej macierzy wszystkich konfiguracji
N05	Ochrona danych	Keystore, HTTPS, kontrola dostępu i walidacja; baza Room bez SQLCipher
N06	Dostępność	Etykiety pól i ikon funkcyjnych, kontrast i większa czcionka; audyt TalkBack pozostaje do wykonania
N07	Testy	Domena Kotlin, backend, Compose UI i integracja REST
N08	Rozbudowa	Typowane modele, wersjonowane operacje, repozytorium i eksport schematu Room
N09	Git i dokumentacja	Historia, README, diagramy, testy, screeny, PDF i APK

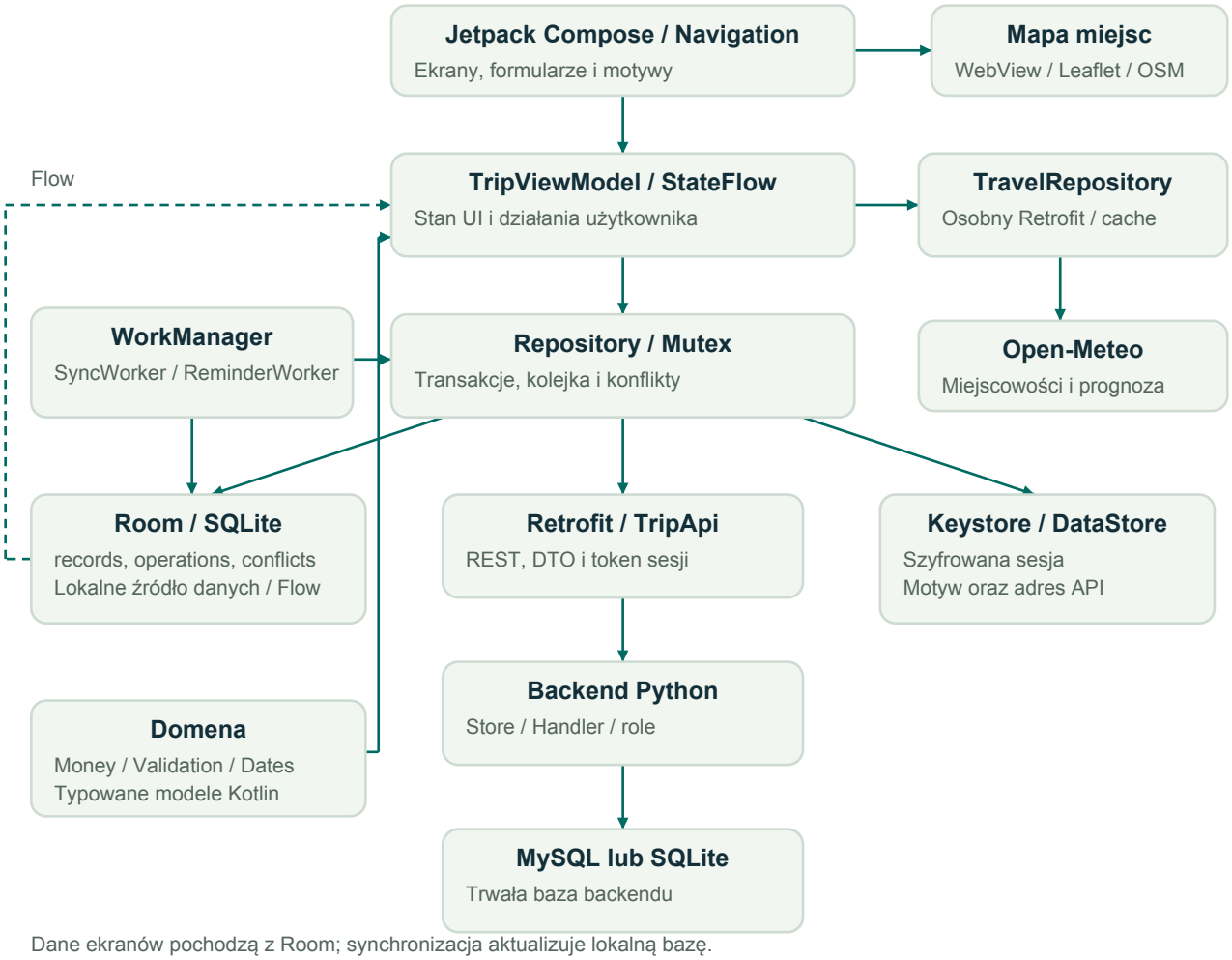
Trwałość i poprawność danych

Room przechowuje dane po ponownym uruchomieniu. Zapis danych i kolejki synchronizacji jest transakcyjny. Walidacja obejmuje m.in. termin wyjazdu, daty punktów planu, godziny, współrzędne i udziały w wydatku. Kwoty są liczbami całkowitymi najmniejszych jednostek waluty; podział kosztów zachowuje sumę co do grosza.

Kontrola dostępu

Backend weryfikuje sesję, członkostwo i rolę przy operacjach. Hasła są hashowane PBKDF2-SHA256 z indywidualną solą (260 000 iteracji); tokeny sesji są przechowywane na serwerze jako skróty. Lokalną sesję chroni AES-GCM i Android Keystore. Kopie zapasowe aplikacji są wyłączone. Lokalna baza Room nie jest szyfrowana; dostęp ogranicza piaskownica systemu Android.

4. Diagram architektury aplikacji



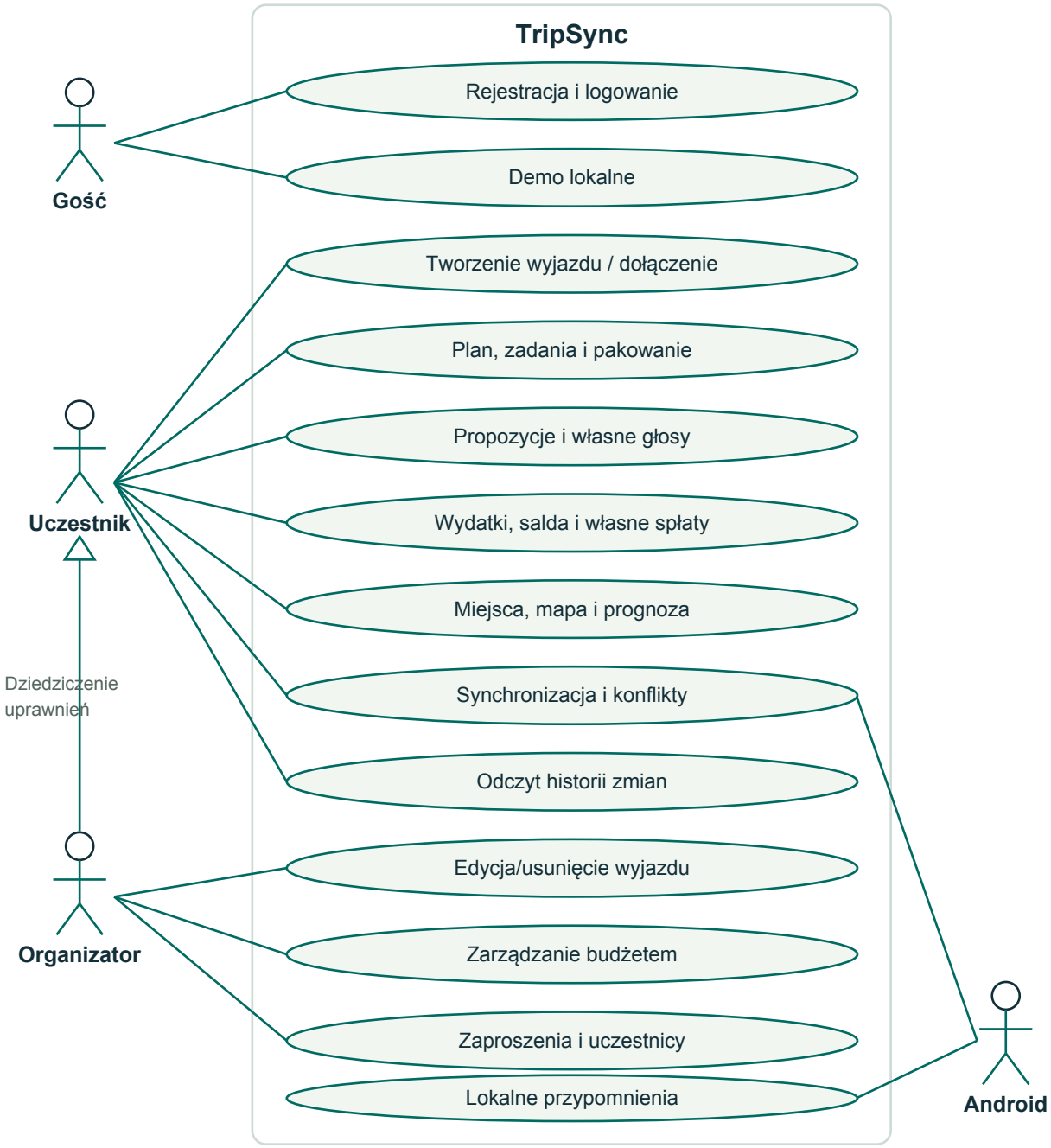
Dane ekranów pochodzą z Room; synchronizacja aktualizuje lokalną bazę.

Aplikacja korzysta z **MVVM**. Compose wyświetla stan z TripViewModel i przekazuje akcje użytkownika. ViewModel udostępnia StateFlow, a Repository koordynuje warstwę danych. Modele, walidacja i obliczenia rozliczeń znajdują się w domenie. Kod jest rozdzielony na foldery ui, domain i data oraz osobny backend.

Room jest lokalnym źródłem danych. Zmiana rekordu oraz wpis do operations powstają w jednej transakcji. Synchronizacja przez Retrofit wysyła operacje z identyfikatorem i wersją bazową. Serwer sprawdza uprawnienia, zapobiega podwójnemu wykonaniu i zwraca konflikty. WorkManager uruchamia synchronizację okresową przy dostępnej sieci; użytkownik może też wywołać ją ręcznie.

Przeniesienie propozycji do harmonogramu jest jedną operacją serwera: zmienia typ rekordu, usuwa głosy i zapisuje historię w transakcji. TravelRepository obsługuje Open-Meteo i cache. Mapę renderuje Leaflet w WebView, a ustawienia przechowuje DataStore.

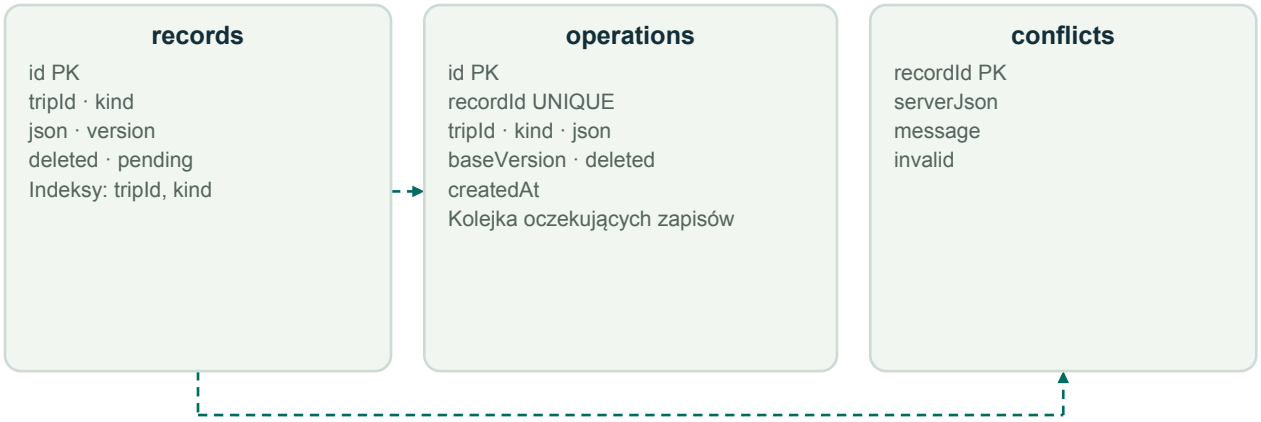
5. Diagram przypadków użycia



Organizator dziedziczy możliwości uczestnika. Gość uruchamia demo, rejestruje się lub loguje. Android realizuje pracę w tle i powiadomienia. Uprawnienia do danych grupy są sprawdzane na serwerze.

Przykładowy przebieg: organizator tworzy wyjazd i udostępnia kod. Uczestnik dołącza, proponuje aktywność i oddaje głos. Propozycja trafia do planu; grupa zapisuje wydatki, sprawdza salda i odnotowuje spłatę. Przy braku sieci zmiany czekają w kolejce.

6. Baza danych na urządzeniu



Powiązania logiczne, bez fizycznych FK. Zapis rekordu i operacji w jednej transakcji Room.

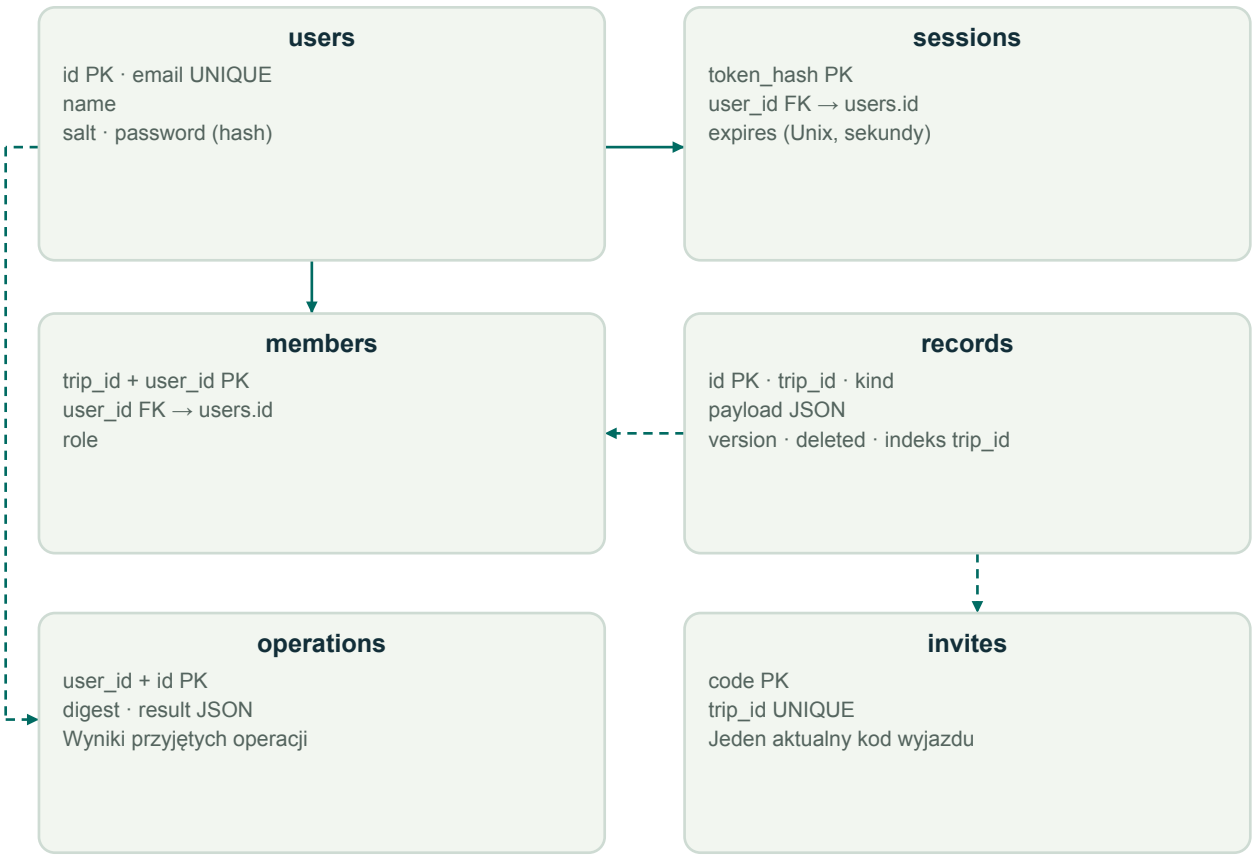
Tabela	Rola w aplikacji
records	Lokalne rekordy wyjazdu i jego elementów. Pole kind określa typ, json zawiera dane modelu, version wersję serwera, pending i deleted stan synchronizacji.
operations	Trwała kolejka zmian: identyfikator operacji, rekord, typ działania, payload, baseVersion i czas utworzenia. Jeden oczekujący wpis na rekord.
conflicts	Wersja serwera i przyczyna konfliktu lub odrzucenia. Użytkownik wybiera zachowanie swojej wersji albo danych serwera.

Relacje i formaty

Diagram pokazuje fizyczne tabele Room/SQLite. Powiązania przerywane są logiczne; schemat nie deklaruje kluczy obcych. records ma indeksy tripId i kind, a operations unikalny recordId. Usunięcia rekordów synchronizowanych wykorzystują znaczniki deleted.

W payloadach znajdują się typowane modele: wyjazd, punkt planu, propozycja, głos, zadanie, wydatek, budżet, spłata, miejsce i wpis historii. Identyfikatory łączą element z wyjazdem oraz użytkownikiem. Daty przechowywane są jako ISO, godziny jako czas 24-godzinny, a kwoty jako liczby całkowite najmniejszych jednostek waluty. Wersja schematu Room: 1.

6. Baza danych backendu



Linia ciągła: fizyczna relacja FK. Linia przerywana: relacja logiczna kontrolowana w kodzie.
MySQL: InnoDB / utf8mb4. SQLite: FK włączone / WAL. Typy szczegółowe: baza-danych.md.

Tabela	Przechowywane dane
users	Konta: identyfikator, nazwa, unikalny e-mail, sól i skrót hasła.
sessions	Skrót tokenu, identyfikator użytkownika i termin wygaśnięcia.
records	Współdzielone rekordy, wyjazd, rodzaj, JSON, wersja i usunięcie.
members	Uczestnictwo i rola; klucz złożony trip_id + user_id.
invites	Kod zaproszenia i unikalny identyfikator wyjazdu.
operations	Wynik operacji i skrót żądania; ponowienie zwraca ten sam wynik.

Backend obsługuje SQLite lokalnie i MySQL na hostingu (InnoDB, utf8mb4). Linie ciągłe oznaczają klucze obce, przerywane powiązania logiczne. Członkostwo, poprawność payloadów i wersje rekordów kontroluje warstwa API.

7. Biblioteki i technologie

Wersje wynikają z deklaracji projektu. Zależności przechodnie rozwiązuje Gradle.

Technologia / wersja	Zastosowanie
Kotlin; plugin Compose 2.2.10	Logika i deklaratywne ekrany. Kotlin wbudowany w AGP.
AGP 9.1.1; Gradle 9.3.1	Gradle Wrapper; Java 17, sprawdzono JDK 21.
Android SDK 36.1	compile 36.1, target 36, minimum 26 (Android 8.0).
Jetpack Compose / Material 3	Motywy, formularze i pickery; BOM 2024.01.00.
Navigation 2.7.7; Lifecycle 2.9.0	Nawigacja, ViewModel i cykl życia.
Room 2.8.4; KSP 2.2.10-2.0.2	SQLite, DAO, transakcje, schemat i obserwacja danych.
Coroutines 1.10.2 / Flow	Operacje asynchroniczne i aktualizacje stanu.
Retrofit 2.9.0; OkHttp 4.12.0; Gson 2.11.0	REST, transport HTTP i serializacja JSON.
DataStore 1.1.7; WorkManager 2.9.0	Ustawienia, synchronizacja i przypomnienia.
Android Keystore / AES-GCM	Ochrona lokalnej sesji.
Python 3.10+; SQLite	Backend REST, lokalna baza i testy.
PyMySQL[rsa] 1.2.3	Połączenie backendu z MySQL na hostingu.
Leaflet 1.9.4 / OpenStreetMap	Mapa w WebView, kafelki HTTPS i atrybucja OSM.
Open-Meteo	Geokodowanie miejscowości i prognoza pogody.
JUnit 4.13.2; AndroidX Test	Testy Kotlin; runner 1.7.0, ext-junit 1.3.0.
Compose UI Test; Espresso 3.7.0	Testy interfejsu i systemowych kontrolek daty.

Leaflet jest dołączony lokalnie z licencją BSD-2-Clause. Mapa pokazuje źródło OpenStreetMap. Aplikacja korzysta z publicznych usług geokodowania i prognozy Open-Meteo. Nie wymaga płatnej subskrypcji.

8. Instalacja i uruchomienie

Instalacja załączonego APK

1. Rozpakuj ZIP i przenieś **TripSync.apk** na telefon z Androidem 8.0+ albo przeciągnij plik do uruchomionego emulatora. Na telefonie zezwól użytej aplikacji na instalowanie APK. Pakiet pl.tripsync ma wersję 1.2.1 i jest podpisany kluczem debug.

2. Otwórz TripSync i wybierz **Wypróbuj bez konta**. Przykładowy wyjazd trzech osób pozwala pokazać edycję planu, zadania, głosy, koszty i budżet bez uruchamiania backendu.

3. Do współpracy ustaw adres API ikoną ustawień przed logowaniem. Wszyscy używają tego samego serwera i własnych kont; dołączają kodem lub linkiem. Adres: **https://tripsync.wisp.uno/**, **bez :9853**. Port 9853 jest portem backendu w panelu; domena HTTPS obsługuje połączenia przez port 443. Wymagany backend API 1.2.0.

Uruchomienie ze źródeł

Pobierz repozytorium Git i otwórz cały katalog w Android Studio. Wymagane: JDK 17 lub 21, SDK 36.1, Build Tools 36.0.0. Zsynchronizuj Gradle, wybierz telefon lub emulator API 26+ i kliknij Run. Budowanie z terminala w katalogu projektu:

```
./gradlew -PtripsyncBuildRoot=/tmp/tripsync-build :app:assembleDebug
```

Wynik: **/tmp/tripsync-build/app/outputs/apk/debug/app-debug.apk**. Na Windows użyj gradlew.bat i własnego katalogu wynikowego.

Własny backend do demonstracji współpracy

Lokalny backend SQLite wymaga Pythona 3.10+; nie wymaga dodatkowych pakietów. W drugim terminalu, w katalogu repozytorium, uruchom:

```
python3 backend/server.py --db /tmp/tripsync-local.sqlite
```

W emulatorze ustaw **http://10.0.2.2:8080/**. Dla telefonu po USB uruchom **adb reverse tcp:8080 tcp:8080** i ustaw **http://127.0.0.1:8080/**. Lokalne HTTP jest dozwolone w wariancie debug; serwer zdalny wymaga HTTPS.

Błąd DNS emulatora przy logowaniu

Jeśli Mac łączy się z API, a Android zgłasza błąd DNS, zamknij emulator i uruchom go ponownie z katalogu repozytorium. Poniższy przykład dotyczy AVD Medium_Phone; użyj nazwy swojego urządzenia. Dane pozostają zachowane.

```
sh scripts/start_emulator.sh Medium_Phone
```

Skrypt ustawia DNS 1.1.1.1 i 8.8.8.8 oraz wykonuje zimny start. Aplikacja rozróżnia błędy DNS, HTTPS, timeout i odmowę połączenia. Nie wyłączaj sprawdzania certyfikatów. Konfiguracja własnego hostingu i MySQL: docs/wispbyte.md w repozytorium.

9. Przeprowadzone testy i wyniki

Weryfikacja z **06.11.2026**: Android 1.2.1, backend API 1.2.0. Pełny przebieg bazowy przed poprawką komunikatów sieciowych:

Grupa	Testy zaliczone	Niepowodzenia
Backend Python	45	0
Jednostkowe Android / Kotlin	23	0
Android: interfejs i integracja	31	0
Łącznie	99	0

Zakres bazowy: walidacja i obliczenia kosztów, konta i role, CRUD, nawigacja, formularze, odtworzenie aktywności, praca dwóch kont, synchronizacja, konflikty, konwersja bez duplikatów, przypomnienia i cache pogody.

Weryfikacja po poprawce połączenia

Grupa	Testy zaliczone	Niepowodzenia
Jednostkowe Android (w tym 7 nowych)	30	0
Połączenie z publicznym API z Androida	2	0

Nowe testy jednostkowe rozróżniają DNS, TLS, timeout, odmowę połączenia i odpowiedzi błędów serwera. Na emulatorze odtworzono UnknownHostException; po poprawieniu DNS sprawdzono /health (200), /sync bez sesji (401) i logowanie z celowo błędnymi danymi (401). Nie tworzono kont i nie zmieniano danych użytkowników.

Zestawy częściowo się pokrywają: 30 testów jednostkowych zawiera wcześniejsze 23. Nie sumujemy obu przebiegów. **Budowanie poprawionego APK:** powodzenie. **Lint offline:** 0 błędów, 1 ostrzeżenie GradleDependency.

Środowisko i granice weryfikacji

Android 10 / API 29, ARM64; SDK 36.1, JDK 21, Python 3.13.5. Bazowe testy używały lokalnej SQLite; pickery sprawdzono też przy 320 dp i czcionce 1.3. Po poprawce wykonano powyższe testy celowane; nie powtarzano pełnej sesji UI. Brak pełnej regresji rzeczywistego MySQL, testów nowszych Androidów, fizycznych urządzeń, TalkBack i obciążenia. Próba z błędnym hasłem nie weryfikuje logowania konkretnym kontem.

Powtórzenie i dowody

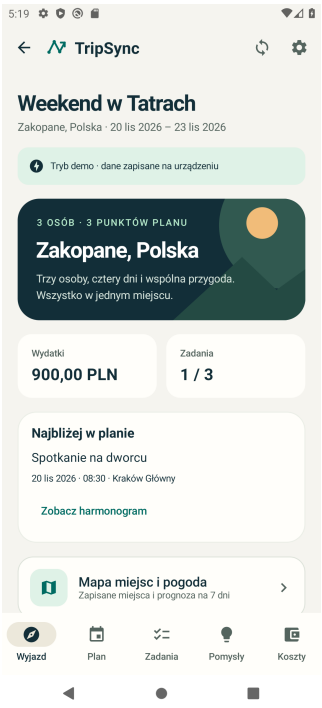
```
python3 -m unittest discover -s backend -v

./gradlew :app:testDebugUnitTest :app:lintDebug :app:connectedDebugAndroidTest
```

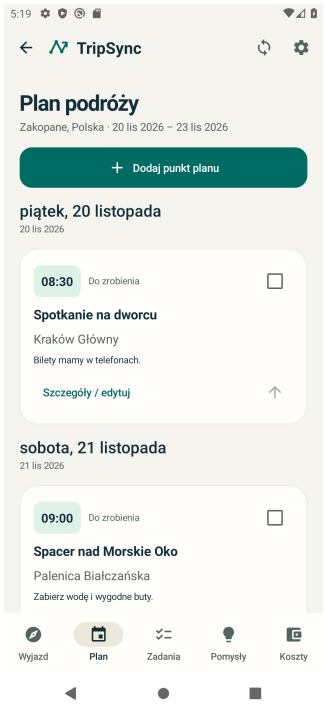
Pełne testy urządzeniowe wymagają osobnego emulatora i lokalnego API pod http://10.0.2.2:8080/. Raporty: przebieg bazowy oraz poprawka połączenia.

10. Najważniejsze widoki (1/2)

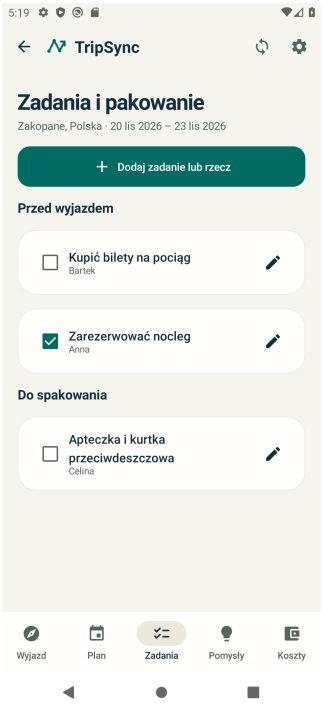
TripSync 1.2.1, emulator Android 10, 06.11.2026. Dane demonstracyjne.



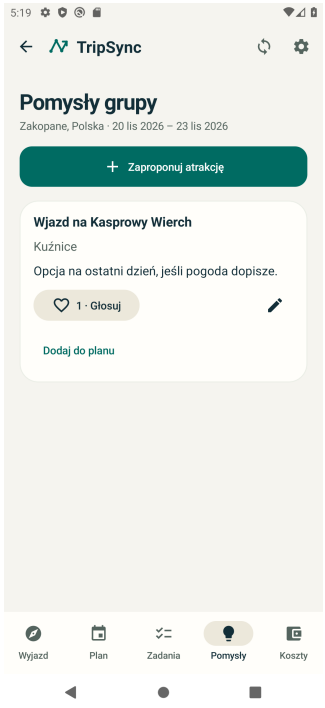
Podsumowanie wyjazdu



Harmonogram



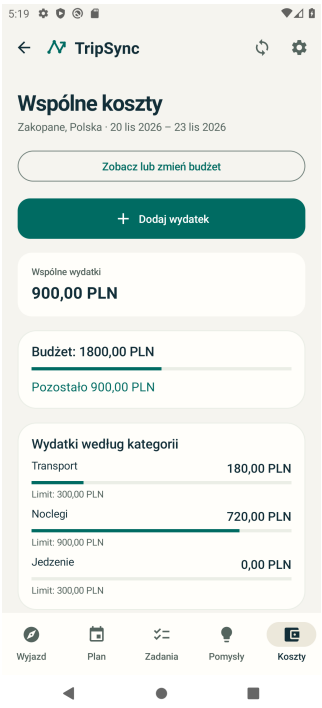
Zadania i pakowanie



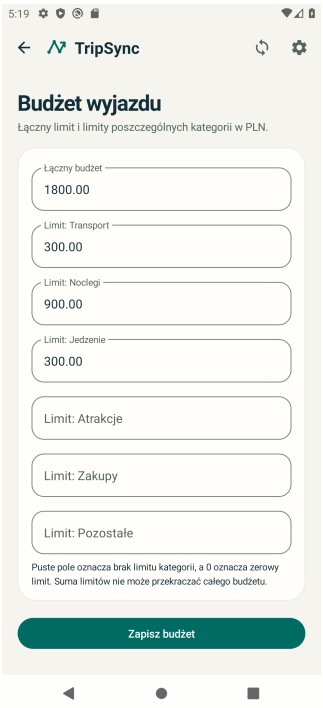
Propozycje i głosowanie

10. Najważniejsze widoki (2/2)

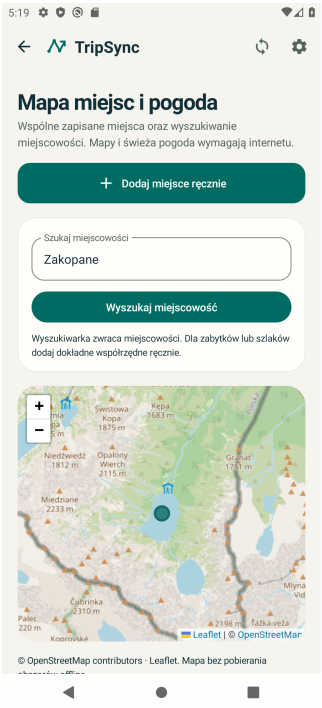
TripSync 1.2.1, emulator Android 10, 06.11.2026. Dane demonstracyjne.



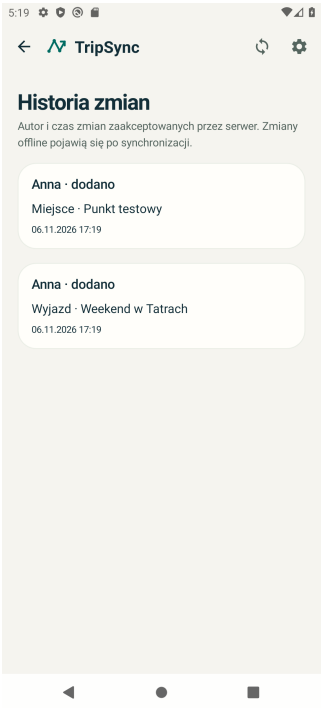
Wydatki i rozliczenia



Budżet i kategorie



Mapa miejsc



Historia zmian